MannMitra: Master Product and Build Spec (v2)

Mental wellbeing screening and 30-day self-help web app. This is a screening and self-help tool, not a diagnosis or treatment.

---

0. Errors found in the original document and how they are

fixed

#	Problem in original	Fix in this spec
1	"Resend triggers the SAME OTP generator" implied the same code is reused	Resend creates a **new** code. The old code stops working. Timer and attempt counter restart.
2	Sign In accepts Email/UserID, but email signup never collects a UserID	Username is collected in `/onboarding` for **both** email and Google users (unique, lowercase).
3	"ONBOARDING (/pretest)" mixed two different steps	Split into `/onboarding` (profile) and `/pretest` (screening).
4	"If flag is down" never defined	Flag is `profiles.onboarding_completed` (false = go to `/onboarding`, true = go to `/home`).
5	"User not found" reveals which emails or usernames exist	One calm message for unknown user **and** wrong password (section 3.3).
6	`/return-404` and `/return-500` as normal URLs	Use the framework's automatic `not-found` and `error` pages. They appear on any bad URL or server error.
7	Forgot Password had no route, and Google-only accounts were not handled	Added `/forgot-password`. Google-only accounts see a message to use Google sign in.
8	"Forgot Username Logic" was empty	Defined in section 3.6 using email OTP.

#	Problem in original	Fix in this spec
9	Google vs email same-email rule had no enforcement method	Defined in section 3.4, including a warning that Supabase may auto-link accounts by default.
10	Screening tool had no scoring and **no self-harm question**	Added scoring (section 5) and a mandatory safety question.
11	Female version tied directly to onboarding	Gender question has a third option. Menstrual module sits behind a feature flag and is built later.
12	Home page blank	Defined in section 13.
13	Unverified signups could linger	Unverified accounts are blocked everywhere and auto-deleted after 24 hours.
14	Result screen, risk gate, track assignment, 30-day program, post-test, report were missing	Added (sections 6 to 12).

---

1. Stack (free tier)

- Next.js (App Router, TypeScript), Tailwind CSS
- Landing page: React Three Fiber (three.js) and Framer Motion, with a lightweight fallback and `prefers-reduced-motion` support
- Supabase: Auth, Postgres with Row Level Security, Storage (drawings, report PDFs)
- Gemini API: AI chatbot (later phase)
- Hosting: Vercel

---

2. Routes and access rules

Route	Who can open it	Notes
`/`	Everyone	3D landing page, Sign Up and Sign In buttons
`/privacy`, `/terms`	Everyone	Legal pages (section 15)
`not-found`, `error`	Automatic	Calm 404 and 500 pages
`/signup`	Logged out	Email + password, or Google
`/signup/verify`	Signed up, email not verified	OTP screen
`/signin`	Logged out	Email or username + password, or Google
`/forgot-password`, `/forgot-username`	Logged out	OTP based
`/onboarding`	Verified, not onboarded	Username, name, gender, consent
`/pretest`	Onboarded, no pretest	MHI-18 + safety question
`/pretest/result`	After pretest	Score, risk gate, track
`/crisis`	Everyone	Helplines and hospital finder
`/home`	Active users	Today's task, progress
`/program`	Active users	30-day map

Route	Who can open it	Notes
`/program/day/[n]`	Active users	Runs that day's task
`/progress`	Active users	Charts, garden, streak
`/posttest`	Day 30 reached	Same screening again
`/report/[programId]`	After posttest	Report with PDF download
`/settings`	Active users	Profile, reminders, export and delete data

Guard logic (middleware)

Each user has one derived stage. The middleware sends them to the only page allowed for that stage.

Stage	Condition	Allowed pages
`unverified`	Email not confirmed	`/signup/verify`
`needs_onboarding`	`onboarding_completed = false`	`/onboarding`
`needs_pretest`	No pretest session	`/pretest`
`blocked`	Pretest severity = severe or safety flag	`/pretest/result`, `/crisis`, `/settings`
`active`	Program running	`/home`, `/program/*`, `/progress`, `/settings`, `/crisis`
`needs_posttest`	Day 30 reached, no posttest	`/posttest`
`completed`	Posttest done	`/report/*`, `/home` (summary), `/settings`

3. Authentication flows

3.1 Rules for every OTP screen

Rule	Value
Code length	6 digits
Code validity	2 minutes
Resend cooldown	2 minutes (button appears when the old code expires)
Wrong attempts	5 per code, then the user must resend
Resend behavior	Generates a new code and cancels the old one
Server enforcement	Set the same 2-minute values in Supabase Auth settings. The screen timer is only for display.
Email template	Must include the code placeholder `{{ .Token }}`, otherwise users get a link instead of a code

3.2 Sign up with email

1. User enters email, password, confirm password (minimum 8 characters).
2. App calls Supabase sign up. "Confirm email" is turned on.
3. Screen always says: "If this email can be registered, we sent a code." (Same text whether or not the email already exists.)
4. `/signup/verify`: OTP input with 2-minute timer.
5. Correct code: email is verified, redirect to `/onboarding`.
6. Wrong code: error, attempt counted. Expired code: "Code expired", Resend appears.
7. Registration is complete **only** after verification. Until then the user can reach nothing else.

8. A scheduled database job deletes unverified accounts older than 24 hours.

3.3 Sign in

1. User enters email **or** username, plus password.
2. If the input has no "@", a server route finds the email for that username (service key kept server side), then signs in.
3. Success: stage decides the redirect (section 2).
4. Failure message (soft white card, calm text, no red): "We couldn't sign you in with those details, but don't worry, you can try again."
5. The same message is used for unknown account and wrong password.
6. If the email exists but is unverified, send the user to `/signup/verify`.
7. Add rate limiting on the server route (for example 10 attempts per 10 minutes per IP).

3.4 Google sign up and sign in

First time with Google (new account)

1. Google returns the verified email and name.
2. Create the account, redirect to `/onboarding`.
3. Name is pre-filled and editable. The user chooses a username. No password is needed.

Returning Google user

1. If `onboarding_completed = true`, go to `/home` (or the stage page).
2. If false, go to `/onboarding`.

Same email already registered with email and password

1. Do not let them in. Show: "An account with this email already exists. Please sign in with your email and password."
2. This stops two accounts merging and losing data.

3. **Important:** Supabase can automatically link a Google login to an existing account with the same email. So the rule must be enforced by code. After the Google callback, check the user's linked identities. If an `email` identity exists, sign the user out, remove the new Google identity (admin API; check Supabase docs for the exact call), and show the message above.
4. Test this case before anything else. It is the riskiest part of auth.

3.5 Forgot password

1. `/forgot-password`: user enters email, taps "Send OTP".
2. Screen always says: "If an account exists, we sent a code."
3. OTP screen with the rules in section 3.1.
4. Correct code creates a temporary session that only allows the next screen.
5. Set new password (minimum 8 characters, confirm). Supabase stores it hashed. Do not write passwords to your own tables.
6. Sign out other devices. The current session continues, so the user goes straight to their stage page.
7. Google-only accounts: show "This account uses Google sign in" instead of sending a code.

3.6 Forgot username

1. `/forgot-username`: user enters email, taps "Send OTP".
2. Same OTP rules as 3.1. Same neutral message.
3. After a correct code, show the username on screen, then sign the temporary session out.
4. Button: "Go to sign in".

3.7 Sessions

- Session stays active until sign out.
- Sign out button in `/settings` and the header.
- Changing the password signs out other devices.

---

4. Onboarding (`/onboarding`)

One screen per step, with a progress bar.

1. Username: 3 to 20 characters, lowercase letters, numbers, underscore, must be unique. Live "available" check.
2. Display name (pre-filled for Google users, editable).
3. Gender: Male, Female, Other or prefer not to say.
    - The menstrual module (female users) is built later and hidden behind a feature flag `cycle_tracking_enabled`.
4. Age confirmation. Decide the target age before launch. If under-18 users are allowed, Indian data protection law may require parental consent, so check with a lawyer.
5. Consent: Terms, Privacy Policy, and "I understand this is not a medical diagnosis" (all required).
6. Optional: daily reminder time.
7. Save, set `onboarding_completed = true`, redirect to `/pretest`.

---

5. Pre-test (`/pretest`)

5.1 Screening tool

Mental Health Inventory (MHI-18), as given in the original document:

- 18 questions, one per screen.
- Prompt: "During the past 4 weeks, how much of the time..."
- Answers: 1 All of the time, 2 Most of the time, 3 A good bit of the time, 4 Some of the time, 5 A little bit of the time, 6 None of the time.
- Items: 1 interesting daily life, 2 depressed, 3 loved and wanted, 4 very nervous, 5 firm control of behavior and emotions, 6 tense or high-strung, 7 calm and peaceful, 8 emotionally stable, 9 downhearted and blue, 10 able to relax, 11 restless or fidgety, 12 moody or brooded, 13 cheerful, 14 low spirits, 15 happy, 16 nothing to look forward to, 17 so down nothing could cheer you up, 18 anxious or worried.

5.2 Mandatory safety question (added)

MHI-18 has no question about self-harm. Add one after item 18:

> Over the last 2 weeks, how often have you had thoughts that you would be better off dead, or of hurting yourself? 0 Not at all, 1 Several days, 2 More than half the days, 3 Nearly every day

This matches the wording of the public PHQ-9 item 9. **Any answer above 0 triggers the risk gate immediately**, whatever the score.

5.3 Scoring

Step 1: recode so a higher number always means better.

- Positive items 1, 3, 5, 7, 8, 10, 13, 15: score = 7 minus the raw answer.
- Negative items 2, 4, 6, 9, 11, 12, 14, 16, 17, 18: score = the raw answer.

Step 2: totals.

- Total = sum of the 18 recoded scores (18 to 108).
- Wellbeing % = (total - 18) / 90 x 100.
- Overall distress % = 100 - wellbeing %.

Step 3: problem areas (provisional mapping).

Area	Items
Anxiety	4, 6, 7, 10, 11, 18
Depression	1, 2, 3, 9, 13, 14, 15, 16, 17
Negative mood	5, 8, 9, 12, 14, 16

For each area: area distress % = 100 - (area wellbeing %), calculated the same way over just those items.

Step 4: severity bands (provisional, on overall distress %).

Distress %	Severity
Under 30	Minimal
30 to 49	Mild
50 to 69	Moderate
70 or more	Severe

> The item-to-area mapping and the band cut-offs above are placeholders built from the item wording. Before launch, replace them with the official MHI scoring key and have a qualified mental health professional review them.

6. Risk gate (`/pretest/result`)

Condition	What the app does
Safety answer above 0	Show the support screen immediately. No program.
Severity = Severe	Show the support screen. No program.
Severity = Moderate	Offer the program, with a gentle recommendation to also see a counsellor.
Severity = Mild	Offer the program.
Severity = Minimal	Show wellness tips and offer the program as optional.

Support screen (calm design, no alarm colors):

- Short kind message: your answers suggest you may benefit from talking to a professional.
- Tele-MANAS helpline: 14416 (India). Emergency: 112.

- Button: find nearby hospitals and mental health centres (Google Places or OpenStreetMap, phone numbers shown).
- No self-help activities are unlocked.
- Store a row in `risk_events`.
- The user can retake the screening only after 14 days. If the safety answer was above 0, show the support screen again each time they sign in.

---

7. Track assignment

1. Compute distress % for anxiety, depression, negative mood.
2. Assign the **highest** area as the track.
3. If the top two are within 10 points, assign the highest and mention the second as a "bonus" suggestion (no blended plan in version 1).
4. Show a result screen: the user's wellbeing %, strongest area, and what the next 30 days look like.
5. Button: "Start my 30 days". This sets `start_date` to today in the user's time zone and creates the program.

---

8. 30-day program engine

Rule	Value
Length	30 days (day 30 = post-test)
Odd days (1, 3 ... 29)	Main activity, 10 to 15 minutes
Even days (2, 4 ... 28)	Break-day micro task, 1 to 3 minutes
Unlock time	Local midnight of `start_date + (day - 1)`
No skipping ahead	A day's task opens only on its day
Catch-up window	A missed task stays open for 48 hours, then becomes `missed`
Late user	Day 30 post-test stays available any time after day 30
Long absence (7+ days)	Friendly welcome back, continue from today, no penalty message
Completion	A task counts as done only after a mood check **before** and **after**

Every task saves: mood before, mood after, time taken, and its own data (for example drawing, journal text).

---

9. Task library

Digital, interactive tasks only. Days 1 to 9 are final. Days 10 to 30 reuse the same components and get written next.

Depression track

Day	Task	Component
1	Pick your activities: drag 5 from a card deck into a ranked list	Drag-and-drop board
2	Water your mood garden and rate mood	Mood check (garden)
3	Build a happy playlist of 5 songs and play one	Media player
4	Mood weather: pick the weather and set strength	Mood check (weather)
5	Laugh break: funny clip and laugh rating	Media player
6	Kind card flip: flip 3 cards, save a favourite	Card flip
7	Make a beat on an 8-step grid	Beat grid
8	Good-thing tap: tap emoji tiles for what went well	Emoji picker
9	Draw your mood on the canvas	Art canvas

Anxiety track

Day	Task	Component
1	Box breathing, 5 minutes	Breathing pacer
2	Calm balloon: hold to inflate, release to float	Breathing pacer (game)
3	Belly breathing: in 4, out 6	Breathing pacer
4	Bubble pop, 1 minute	Bubble game
5	Scene grounding: tap items in an illustrated room	Grounding scene
6	Squeeze and release: hold to fill, release to drain	Tension slider
7	Muscle relaxation audio with body outline	Media player with body outline
8	Pick a calm sound and volume	Sound mixer
9	Worry sort: drag worries into "I can control" or "I can't"	Drag-and-drop board

Negative thinking track

Day	Task	Component
1	Thinking-trap game: match 4 thoughts to traps	Drag-and-drop board
2	Trap spotter: 3 quick sentences	Quick quiz
3	Thought record: situation, thought, evidence for and against, balanced thought	Thought record form
4	Word swap: replace "should", "always", "never" in 5 sentences	Word swap
5	Gratitude journal: 3 good things and why	Journal form
6	Kind card flip with self-kind sentences	Card flip
7	Thought cloud: draw and redraw kinder	Art canvas (redraw mode)
8	What went well: tap emoji tiles	Emoji picker
9	Friend reframe in a chat-style screen	Journal form (chat skin)

Day 10 to 30 outline (to be written next)

- Depression: more behavior activation (learn-a-skill mini games, gratitude, social message drafts, future-self letter, planning).
- Anxiety: fear ladder (build and rate steps), worry time, body scan audio, thought challenge, toolkit.
- Negative thinking: more thought records, strengths list, self-compassion letter, best-possible-self writing, wins jar, belief experiment tracker.

10. Reusable components

Component	Used by	Notes
Mood check (0 to 10, with garden or weather skins)	Every task	Required before and after
Task card (text, example, timer, done, celebration)	Every task	Shell around the others
Drag-and-drop board	Depression 1, anxiety 9, negative thinking 1	Same engine, different labels and bins
Card flip	Depression 6, negative thinking 6	Only card text changes
Emoji picker	Depression 8, negative thinking 8	Only tile set changes
Art canvas	Depression 9, negative thinking 7	Redraw layer is an option
Media player (audio, YouTube, body outline option)	Depression 3 and 5, anxiety 7	Timer built in
Journal form (chat skin option)	Negative thinking 5 and 9	Saved to timeline
Breathing pacer (circle, balloon modes)	Anxiety 1 to 3	Anxiety only
Thought record form	Negative thinking 3 and later	Reused later by anxiety

Component	Used by	Notes
Beat grid, bubble game, grounding scene, tension slider, quick quiz, word swap, sound mixer	One track each	Kept separate on purpose

---

11. Progress and dashboard (`/progress`, shown in part on `/home`)

- Program progress ring: days completed out of 30.
- Completion rate: done tasks / due tasks.
- Mood chart: daily average of mood before and after.
- Task effect list: tasks with the biggest improvement (mood after minus mood before).
- Gentle streak: consecutive days with any completed task. One missed day does not reset it (one "grace day" per week). Never use shaming messages.
- Mood garden: grows in 6 stages as days complete.
- Pretest vs current wellbeing %, shown after the post-test.

---

12. Post-test and report

Post-test (`/posttest`, opens on day 30)

- Same MHI-18 and the same safety question.
- If the safety answer is above 0 or severity is severe, show the support screen first.

Report (`/report/[programId]`)

Generated automatically after the post-test and downloadable as PDF.

1. Summary: start and end dates, tasks completed, completion rate.
2. Scores: pretest vs post-test wellbeing %, overall and for each area, with the change.

3. Result label per area: improved, no clear change, or worse. A change of 5 points or more in wellbeing % counts as meaningful (provisional cut-off).
4. Mood trend: average mood first 7 days vs last 7 days.
5. Best-working activities: top 3 by mood improvement.
6. Next steps: continue with favourite activities, or see a professional if scores stayed high or got worse.
7. Disclaimer: this is a self-help screening tool, not a diagnosis.

---

13. Home page (`/home`)

- Header: logo, crisis help button (always visible), profile menu.
- Today's task card (or a "rest day" micro card), with a start button.
- Progress ring and gentle streak.
- Quick mood check.
- Mood garden preview.
- Next unlock countdown.
- Later phase: AI chatbot and nearby hospital finder.

---

14. Database (Supabase Postgres)

```sql
```

```sql
create extension if not exists pgcrypto;

-- Profile, one per auth user
create table profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  username text,
  display_name text,
  gender text check (gender in ('male','female','other')),
  auth_provider text check (auth_provider in ('email','google')),
  onboarding_completed boolean not null default false,
  consent_accepted_at timestamptz,
  timezone text not null default 'Asia/Kolkata',
  cycle_tracking_enabled boolean not null default false,
  reminder_time time,
  created_at timestamptz not null default now(),
  constraint username_format check (username is null or username ~ '^
);
create unique index profiles_username_key on profiles (lower(username

-- Create the profile row automatically when an auth user is created
create function handle_new_user() returns trigger
language plpgsql security definer set search_path = public as $$
begin
  insert into profiles (id, display_name, auth_provider)
  values (new.id,
          new.raw_user_meta_data->>'full_name',
          coalesce(new.raw_app_meta_data->>'provider','email'));
  return new;
end $$;
create trigger on_auth_user_created after insert on auth.users
for each row execute function handle_new_user();

-- Screening sessions (pre and post)
create table screening_sessions (
  id uuid primary key default gen_random_uuid(),
```

```
  user_id uuid not null references profiles(id) on delete cascade,
  kind text not null check (kind in ('pre','post')),
  safety_answer int not null check (safety_answer between 0 and 3),
  total_score int,
  wellbeing_pct numeric,
  distress_pct numeric,
  anxiety_pct numeric,
  depression_pct numeric,
  negmood_pct numeric,
  severity text check (severity in ('minimal','mild','moderate','seve
  completed_at timestamptz not null default now()
);

create table screening_answers (
  session_id uuid references screening_sessions(id) on delete cascade
  item_no int check (item_no between 1 and 18),
  raw_value int not null check (raw_value between 1 and 6),
  primary key (session_id, item_no)
);

-- The 30-day program
create table programs (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references profiles(id) on delete cascade,
  track text not null check (track in ('depression','anxiety','negati
  start_date date not null,
  status text not null default 'active'
    check (status in ('active','completed','abandoned')),
  pre_session_id uuid references screening_sessions(id),
  post_session_id uuid references screening_sessions(id),
  created_at timestamptz not null default now()
);
-- Only one active program per user
create unique index one_active_program on programs (user_id) where st

-- Task library (same for everyone, read only)
```

```sql
create table task_templates (
  id uuid primary key default gen_random_uuid(),
  track text not null,
  day_no int not null check (day_no between 1 and 29),
  title text not null,
  description text,
  example_text text,
  component text not null,
  duration_min int,
  config jsonb not null default '{}',
  unique (track, day_no)
);

-- What the user did each day
create table task_completions (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references profiles(id) on delete cascade,
  program_id uuid not null references programs(id) on delete cascade,
  day_no int not null,
  status text not null default 'locked'
    check (status in ('locked','available','done','missed')),
  mood_before int check (mood_before between 0 and 10),
  mood_after int check (mood_after between 0 and 10),
  payload jsonb not null default '{}',
  completed_at timestamptz,
  unique (program_id, day_no)
);

create table mood_entries (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references profiles(id) on delete cascade,
  program_id uuid references programs(id) on delete set null,
  entry_date date not null default current_date,
  mood int not null check (mood between 0 and 10),
  created_at timestamptz not null default now()
);
```

```sql
create table journal_entries (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references profiles(id) on delete cascade,
  program_id uuid references programs(id) on delete cascade,
  day_no int,
  kind text not null,
  content text not null,
  created_at timestamptz not null default now()
);

create table thought_records (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references profiles(id) on delete cascade,
  program_id uuid references programs(id) on delete cascade,
  day_no int,
  situation text,
  thought text,
  feeling_before int check (feeling_before between 0 and 100),
  evidence_for text,
  evidence_against text,
  balanced_thought text,
  feeling_after int check (feeling_after between 0 and 100),
  created_at timestamptz not null default now()
);

create table drawings (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references profiles(id) on delete cascade,
  program_id uuid references programs(id) on delete cascade,
  day_no int,
  title text,
  storage_path text not null,          -- private bucket: drawings/{u
  created_at timestamptz not null default now()
);
```

```sql
-- Songs, beats, favourite cards
create table saved_items (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references profiles(id) on delete cascade,
  kind text not null check (kind in ('song','beat','kind_card','strer
  data jsonb not null,
  created_at timestamptz not null default now()
);

create table risk_events (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references profiles(id) on delete cascade,
  session_id uuid references screening_sessions(id),
  reason text not null check (reason in ('safety_answer','severe_scor
  created_at timestamptz not null default now()
);

create table reports (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references profiles(id) on delete cascade,
  program_id uuid not null references programs(id) on delete cascade,
  summary jsonb not null,
  pdf_path text,
  generated_at timestamptz not null default now()
);

-- Row Level Security: each user sees only their own rows
alter table profiles enable row level security;
create policy "own profile" on profiles
  for all using (id = auth.uid()) with check (id = auth.uid());

do $$
declare t text;
begin
  foreach t in array array[
    'screening_sessions','programs','task_completions','mood_entries'
```

```
    'journal_entries','thought_records','drawings','saved_items',
    'risk_events','reports'
  ] loop
    execute format('alter table %I enable row level security', t);
    execute format(
      'create policy "own rows" on %I for all using (user_id = auth.u
  end loop;
end $$;

alter table screening_answers enable row level security;
create policy "own answers" on screening_answers for all
  using (exists (select 1 from screening_sessions s
                 where s.id = session_id and s.user_id = auth.uid()));

alter table task_templates enable row level security;
create policy "read templates" on task_templates
  for select using (auth.role() = 'authenticated');

-- Remove unverified accounts after 24 hours (schedule with pg_cron)
-- delete from auth.users where email_confirmed_at is null and create
```

Notes:

- Passwords live only in Supabase's own auth tables, hashed. Never add a password column to your tables.
- Scoring and risk-gate checks should run on the server, not only in the browser.
- The username lookup used at sign in runs on a server route with the service key. The service key must never reach the browser.
- Drawings go in a private storage bucket with a policy limiting each user to their own folder.

---

15. Security, privacy, and legal

- Treat all screening answers, journals, and drawings as sensitive health data.
- Row Level Security on every table (section 14). Test it by trying to read another user's rows.
- Never log OTP codes or passwords.
- Rate limit sign in, OTP sending, and OTP verification.
- Settings page: export my data and delete my account (cascade delete removes everything).
- `/privacy` must cover: what is collected (email, name, answers, journals, drawings, mood), why, how long it is kept, who can see it (only the user), how to delete it, third parties (Supabase, Google sign in, Gemini), and a contact address.
- `/terms` must cover: not a medical service or diagnosis, not for emergencies, helplines, age rule, acceptable use, limitation of liability, and changes to terms.
- Have both pages reviewed by a lawyer who knows Indian data protection law before launch. A generated draft is a starting point, not legal clearance.
- Plain, calm wording in terms and privacy pages, with short summaries at the top of each section.

---

16. Build phases

Phase	Work
1	Project setup, Supabase project, tables and RLS, middleware stage guard
2	Auth: email signup with OTP, sign in, Google with the same-email check, forgot password, forgot username
3	Onboarding, pretest, scoring, risk gate, result screen, crisis page
4	Program engine, home page, mood check, task card, days 1 to 9 components
5	Progress dashboard, garden, streak
6	Post-test and report with PDF
7	Days 10 to 30 content
8	Landing page 3D design, 404 and 500 pages, privacy and terms
9	Chatbot and hospital finder
10	Menstrual module (female version), smartwatch ideas

Commit and push to GitHub after each phase.